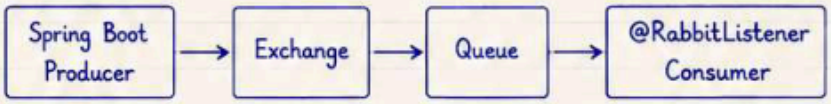
KAFKA & RABITMQ HANDBOOK

From Zero To Hero Notes

1. RABBITMQ WITH SPRING BOOT (OVERVIEW)

- Spring Boot makes it easy to integrate RabbitMQ.
- Steps:
  1. Add dependency
  2. Configure connection
  3. Declare Exchange
  4. Declare Queue
  5. Bind Queue with Exchange
  6. Send messages (Producer)
  7. Receive messages (Consumer)

SPRING BOOT FLOW



@Configuration	→	Define beans (Exchange, Queue, Binding)
@Bean	→	Create and configure beans
RabbitTemplate	→	Send messages to Exchange
@RabbitListener	→	Consume messages from Queue
ConnectionFactory	→	Manage connection to RabbitMQ

2. RABBITMQ vs KAFKA

Aspect	RabbitMQ	Kafka
Type	Message Broker	Event Streaming Platform
Communication	Queue based	Log / Partition based
Routing	Strong routing via Exchanges	Topic & Partition based
Message Handling	Consumed & Acknowledged	Retained for some time
Throughput	Medium	Very High
Ordering	Per Queue	Per Partition
Use Case	Task queues, jobs, commands	Event streaming, logs, analytics

3. REAL-WORLD USE CASES

- Order processing
- Email / SMS notifications
- Payment processing
- Background jobs
- Microservice communication
- Retry workflows
- File processing
- Report generation
- Chat applications
- IoT device data handling

4. RABBITMQ INTERVIEW QUESTIONS (QUICK RECAP)

- What is RabbitMQ?
- What is AMQP?
- What are Exchanges? Types?
- What are Queues? Types?
- What is Binding?
- What is Routing Key?
- Difference between Direct, Topic and Fanout?
- What is ACK? Types of ACK?
- What is NACK?
- What is message durability?
- What is persistent message?
- What is Prefetch and why it is used?
- What is Dead Letter Queue (DLQ)?
- How does RabbitMQ handle failed messages?
- What is TTL in RabbitMQ?
- How does Retry mechanism work?
- What happens if consumer is down?
- How RabbitMQ handles high traffic?
- RabbitMQ vs Kafka
- Use cases of RabbitMQ

5. RABBITMQ CHEAT SHEET

Exchanges	Queues	Acknowledgement	Reliability	Key Terms
• Direct	• Durable	• Auto ACK	• Durable Exchange	• Producer
• Fanout	• Temporary	• Manual ACK	• Durable Queue	• Exchange
• Topic	• Exclusive	• ack()	• Persistent Messages	• Binding
• Headers	• Auto-delete	• nack()	• Prefetch	• Queue
		• reject()	• Retry	• Consumer
			• DLQ	• Message

Pro Tip: Use proper acknowledgement, durability and DLQ to build reliable and fault-tolerant messaging systems.



KAFKA & RABITMQ HANDBOOK

From Zero To Hero Notes

3. RELIABILITY, ACKNOWLEDGEMENT & DURABILITY

1. MESSAGE ACKNOWLEDGEMENT

- Producer sends message to Queue.
- Consumer receives message.
- After successful processing, Consumer sends acknowledgement to RabbitMQ.
- If processing fails, Consumer can reject or not acknowledge the message.

Type	Description	When to Use
Auto ACK	Broker considers message acknowledged as soon as it is delivered.	When message loss is acceptable.
Manual ACK	Consumer explicitly sends ack after processing.	When reliability is important.
NACK	Consumer notifies failure and message can be requeued or sent to DLQ.	When processing fails.
Reject	Similar to NACK but without requeue option.	When you don't want to retry.

2. DURABILITY & PERSISTENCE

- Durable Exchange → Survives broker restart.
- Durable Queue → Survives broker restart.
- Persistent Messages → Stored to disk before acknowledged.

Benefit: No message loss in case of broker restart or crash.

3. PREFETCH (QOS)

- Prefetch defines how many messages a consumer can receive without acknowledging.
- Helps in fair distribution and prevents consumers from being overloaded.

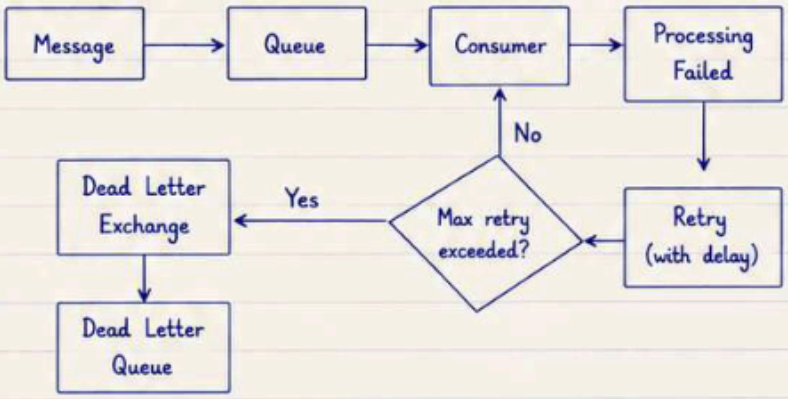
Example

Prefetch = 1 → One message at a time
Prefetch = 5 → Five messages at a time
Prefetch = N → N messages at a time

4. RETRY & DEAD LETTER EXCHANGE (DLX)

Messages go to DLQ when:

1. Rejected by consumer.
2. TTL (Time To Live) expired.
3. Max retry attempts exceeded.
4. Queue length limit exceeded.



5. FAILURE SCENARIOS

- If consumer is down → Messages remain in Queue.
- If consumer processing fails → Message can be retried.
- If message cannot be processed → Send to DLQ.
- If broker restarts → Durable setup ensures no data loss.

KEY TAKEAWAYS

- Acknowledgement ensures reliable delivery.
- Durability + Persistence prevents data loss.
- Prefetch improves performance and fairness.
- DLQ helps handle failed messages safely.

KAFKA & RABITMQ HANDBOOK

From Zero To Hero Notes

2. EXCHANGES, QUEUES & MESSAGE FLOW

1. RABBITMQ MESSAGE FLOW

Producer ⟶ Exchange ⟶ Binding ⟶ Queue ⟶ Consumer

2. EXCHANGE TYPES

- Direct Exchange → Routes messages to queues with exact routing key.
- Fanout Exchange → Broadcasts messages to all bound queues.
- Topic Exchange → Routes messages based on pattern (wildcards).
  Example: order.created , order.* , *.created
- Headers Exchange → Routes based on message headers instead of routing key.

3. ROUTING KEY

- A routing key is used by Exchange to determine the destination Queue.
- Example routing keys:

  order.created payment.success inventory.update

4. QUEUE CONCEPTS

Durable Queue	Survives broker restart.
Temporary Queue	Exists only while the connection is open.
Exclusive Queue	Used by one connection and one consumer only.
Auto-delete Queue	Deleted automatically when last consumer unsubscribes.

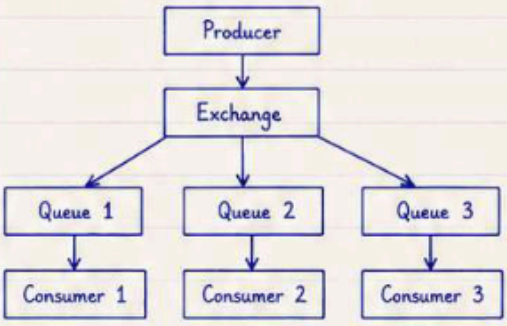
5. EXCHANGE TO QUEUE BINDING

- Binding connects Exchange to Queue with a routing key (or rules).
- One Exchange can be bound to multiple Queues.
- One Queue can be bound to multiple Exchanges.

6. EXCHANGE TYPES SUMMARY

Exchange Type	Description
Direct	Routes message to queue with exact routing key.
Fanout	Routes message to all bound queues.
Topic	Routes message based on routing key pattern.
Headers	Routes message based on headers.

7. MESSAGE FLOW DIAGRAM



KEY TAKEAWAY:

- Exchange decides where the message goes.
- Queues store messages until consumers read them.

KAFKA & RABITMQ HANDBOOK

From Zero To Hero Notes

1. WHAT IS RABBITMQ?

- RabbitMQ is an open-source message broker.
- It implements the AMQP (Advanced Message Queuing Protocol) standard.
- It allows applications to communicate asynchronously by sending messages.
- Producer sends messages → Broker (RabbitMQ) → Consumer receives messages.

BASIC IDEA



- Decouples Producer & Consumer
- Stores messages until consumed
- Ensures reliable delivery

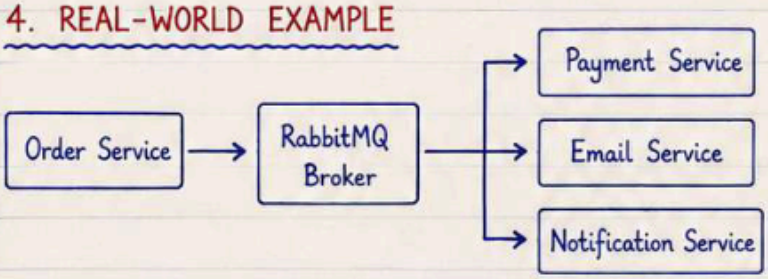
2. WHY RABBITMQ?

- ✓ Asynchronous communication
- ✓ Reliable message delivery
- ✓ Decouples applications
- ✓ Handles traffic spikes with queuing
- ✓ Supports routing, retry & failure handling
- ✓ High availability & scalability

3. PROBLEMS WITH DIRECT COMMUNICATION

- ✗ Tight coupling between services
- ✗ If consumer is down, producer fails or loses data
- ✗ No message buffering
- ✗ Difficult to handle high traffic
- ✗ No retry or failure handling mechanism

4. REAL-WORLD EXAMPLE



When an order is placed, Order Service sends a message to RabbitMQ. Multiple services can consume this message independently.

5. CORE COMPONENTS OVERVIEW

Producer	Application that sends messages to RabbitMQ.
Exchange	Receives messages from producers and routes them to queues.
Queue	Stores messages until they are consumed.
Binding	Connects Exchange to Queue with routing rules.
Consumer	Application that consumes (reads) messages from queues.
Message	The data packet that is transferred.

PRO TIP: RabbitMQ is best suited for task queues, background jobs, and reliable message delivery.

@ Aman_Views

KAFKA HANDBOOK

Kafka From Zero To Hero Notes

1. WHAT IS KAFKA?

Apache Kafka is a distributed event streaming platform used for building real-time data pipelines and streaming applications.

2. WHY KAFKA?

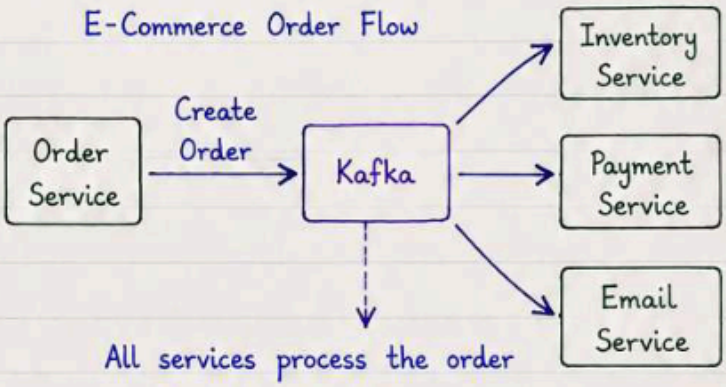
- ✓ High Throughput
- ✓ Scalable & Distributed
- ✓ Fault Tolerant
- ✓ Real-time Data Processing
- ✓ Durable & Reliable
- ✓ Decouples Systems

3. PROBLEMS WITH TRADITIONAL COMMUNICATION

- X Systems are tightly coupled.
- X If consumer is down, producer fails.
- X No buffer for high volume of data.
- X Hard to scale.
- X No real-time data processing.

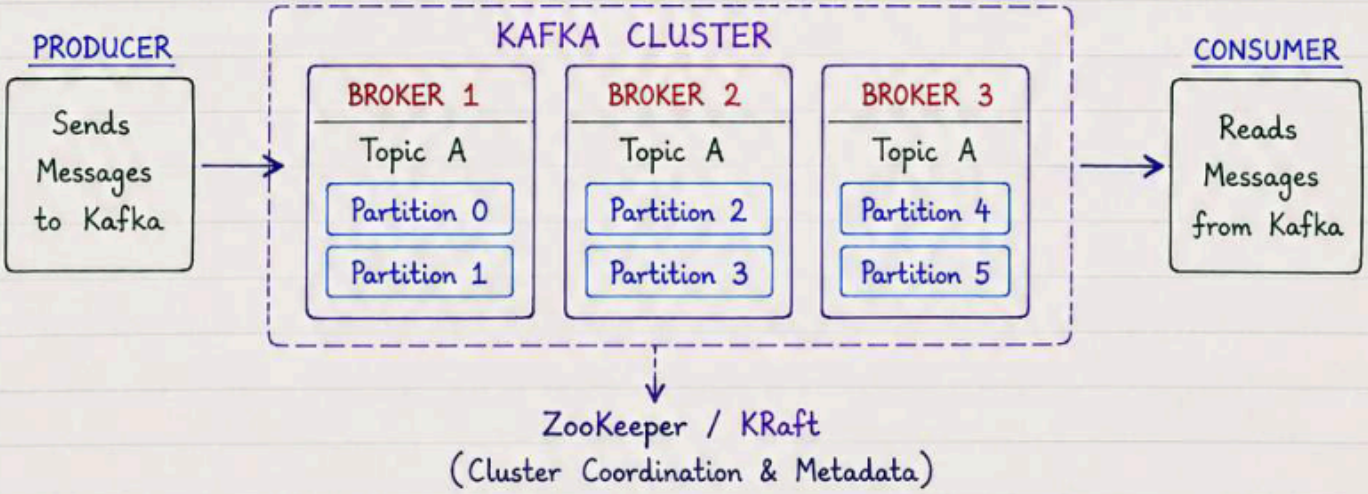
4. REAL-WORLD EXAMPLE

E-Commerce Order Flow



All services process the order event independently.

5. CORE COMPONENTS OVERVIEW



ZooKeeper / KRaft
(Cluster Coordination & Metadata)

6. COMPANIES USING KAFKA

Netflix | Uber | LinkedIn | Swiggy | Amazon | Zomato

NOTE Kafka is not just a message queue, it is a distributed event streaming platform. It stores data, processes data, and delivers data in real-time.

KAFKA HANDBOOK (2/8)

@ aman_views

2. KAFKA ARCHITECTURE

Kafka follows a distributed, partitioned and replicated architecture to achieve high throughput, scalability and fault tolerance.

CORE COMPONENTS

① **BROKER**
- A Kafka server that stores data and handles requests.

② **TOPIC**
- A logical channel to which producers publish messages.

③ **PARTITION**
- A topic is split into multiple partitions. Each partition is ordered and immutable.

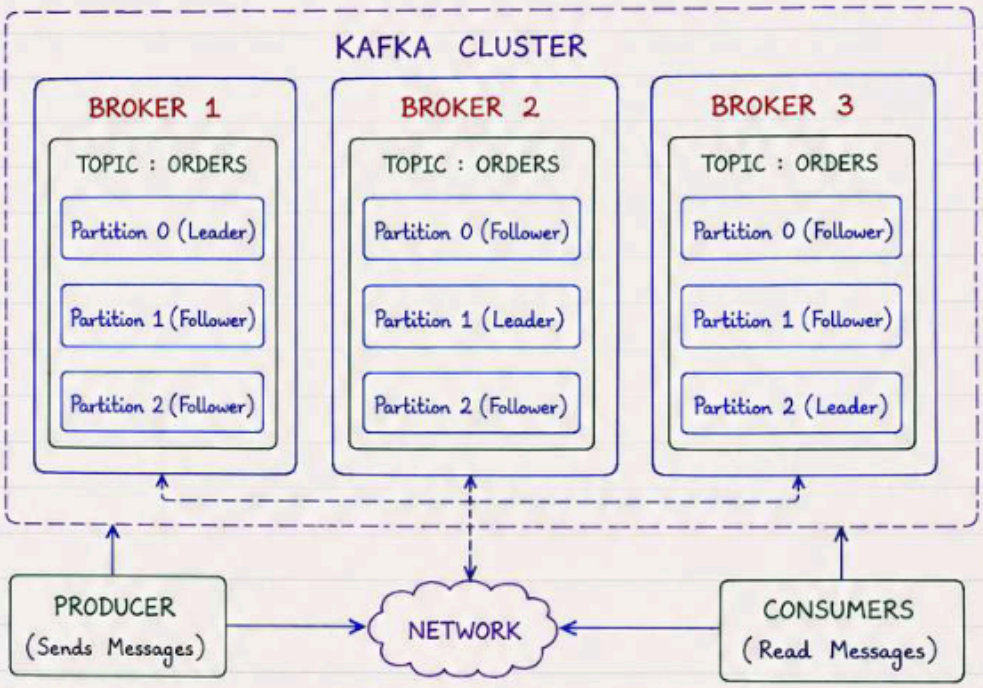
④ **OFFSET**
- A unique number assigned to each message within a partition.

⑤ **REPLICATION**
- Partitions are replicated across multiple brokers for reliability.

⑥ **ZOOKEEPER (OLD)**
- Used for broker coordination and metadata management. (In new versions replaced by KRaft)

⑦ **KRaFT (NEW)**
- Kafka Raft metadata mode. No need of ZooKeeper. Built on Raft consensus.

ARCHITECTURE DIAGRAM



HOW IT WORKS ?

① Producer publishes messages to a Topic.
② Topic is divided into multiple Partitions.
③ Each Partition has a Leader and Followers (replicas).
④ Consumers read messages from Partitions.
⑤ Offsets keep track of messages.
⑥ If Leader fails, a Follower becomes new Leader.

EXAMPLE

Topic : Orders

Partition 0 → Leader (Broker 1)
Replicas → Broker 2, Broker 3

Partition 1 → Leader (Broker 2)
Replicas → Broker 1, Broker 3

Partition 2 → Leader (Broker 3)
Replicas → Broker 1, Broker 2

KEY BENEFITS

- ✓ High Throughput
- ✓ Scalability (Horizontal)
- ✓ Fault Tolerance
- ✓ Durability (Data Replication)
- ✓ Decouples Systems
- ✓ Real-time Data Streaming

OLD vs NEW (ZOOKEEPER vs KRaFT)

FEATURE	ZOOKEEPER (OLD)	KRaFT (NEW)
Metadata Management	Handled by ZooKeeper	Handled by Kafka itself
Extra Dependency	Yes (Needs ZooKeeper)	No (Built-in)
Performance	Slightly slower	Better
Setup	Complex	Simpler
Consensus	External	Raft Consensus (Internal)

NOTE :
* A Topic can have many Partitions.
* Each Partition is assigned to one Leader and multiple Followers.

REMEMBER
Partition = Parallelism
Replication = Reliability

3. PRODUCER & CONSUMER

Kafka follows a simple yet powerful model :



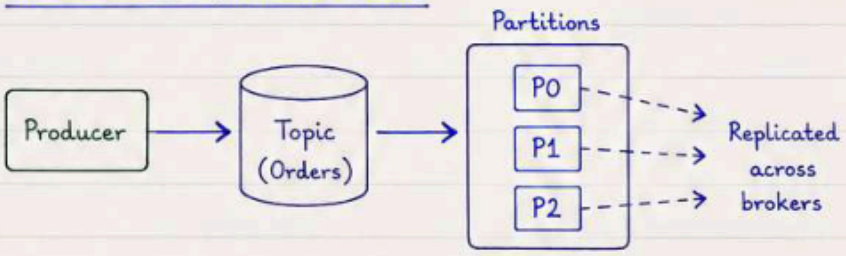
3.1 PRODUCER

① Producer creates a message.
② It sends the message to a Topic.
③ Kafka decides the Partition (based on key or round-robin).
④ Message is stored in the Partition by the Leader.

KEY POINTS

✓ Key helps in deciding the Partition.
✓ Same key always goes to same Partition.
✓ If no key → messages are distributed in round-robin manner.

MESSAGE FLOW (PRODUCER SIDE)



JAVA PRODUCER EXAMPLE (Spring Boot)

```
@Autowired
private KafkaTemplate<String, String> kafkaTemplate;
public void sendMessage() {
    String topic = "orders-topic";
    String key = "order-123";
    String value = "Order Created";

    // send message
    kafkaTemplate.send(topic, key, value);
}
```

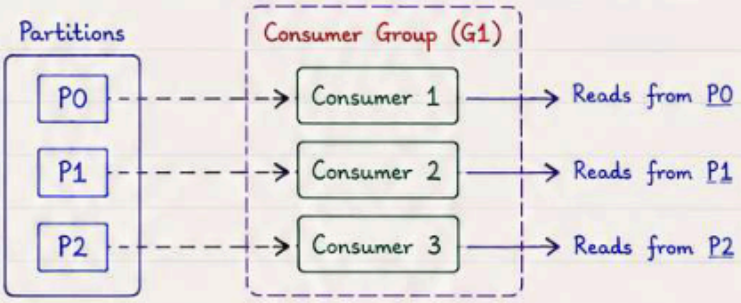
3.2 CONSUMER

① Consumer subscribes to one or more Topics.
② It belongs to a Consumer Group.
③ It pulls (polls) messages from Kafka.
④ It processes the message.
⑤ It commits the Offset after successful processing.

KEY POINTS

✓ Partitions are distributed among consumers within the same group.
✓ If a consumer is down, others will rebalance and take over its partitions.
✓ Consumer Group → enables scaling and fault tolerance.

MESSAGE FLOW (CONSUMER SIDE)



JAVA CONSUMER EXAMPLE (Spring Boot)

```
@KafkaListener(topics = "orders-topic", groupId = "order-group")
public void listen(String message) {
    System.out.println("Received : " + message);

    // Process message
}
```

IMPORTANT TERMS

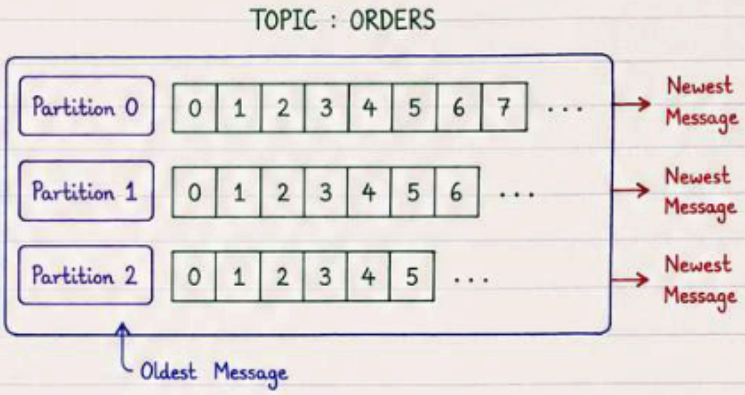
- **Key** : Used to decide Partition.
- **Value** : Actual message (payload).
- **Topic** : Logical channel.
- **Partition** : Parallel log within a Topic.
- **Offset** : Unique ID of message in a Partition.
- **Consumer Group** : Group of consumers working together.
- **Commit** : Saving the offset after processing.

Producer → Writes messages to Kafka. | Consumer → Reads messages from Kafka.

4. TOPICS, PARTITIONS & CONSUMER GROUPS

* **TOPIC** : A topic is a category or feed name to which messages are published.

* **PARTITION** : A topic is split into multiple partitions. Each partition is an ordered, immutable sequence of messages.



WHY PARTITIONS ?

- ✓ Enables parallelism (more consumers can read simultaneously).
- ✓ Increases throughput.
- ✓ Allows horizontal scaling.
- ✓ Maintains order within each partition.
- ✓ Improves fault tolerance.

IMPORTANT POINTS

- Order is guaranteed only within a partition.
- Messages in different partitions may be processed out of order.
- More partitions → more parallelism.
- Too many partitions → overhead.

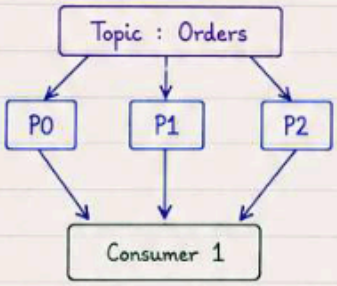
CONSUMER GROUP : A group of consumers that work together to consume messages from a topic.

RULE : In a consumer group, one partition can be consumed by only one consumer at a time.

HOW CONSUMER GROUPS WORK ?

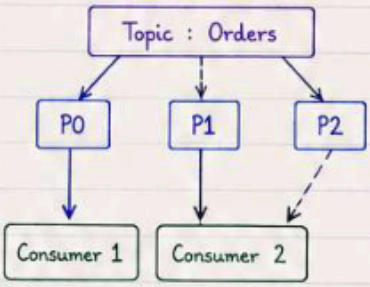
CASE 1 : 1 CONSUMER (G1)

All partitions are read by one consumer.



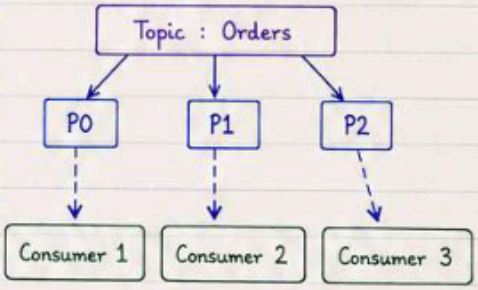
CASE 2 : 2 CONSUMERS (G1)

Partitions are distributed among consumers.



CASE 3 : 3 CONSUMERS (G1)

Each consumer reads from different partition.



REBALANCING

- When a consumer joins or leaves a group, Kafka automatically rebalances partitions.
- Partitions are reassigned to active consumers.
- This ensures high availability and efficient utilization.

EXAMPLE :

Topic has 3 partitions (P0, P1, P2)
Consumer Group (G1)

- If 2 consumers are active → p0 to C1, p1 to C2, p2 to C1
- If 1 consumer leaves → Rebalance happens and remaining consumer takes over partitions.

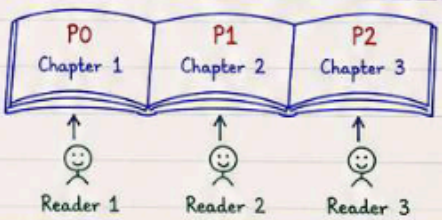
KEY TAKEAWAYS

- ✓ More partitions → higher throughput.
- ✓ Order is guaranteed only inside partition.
- ✓ Consumer Group enables parallel and fault-tolerant processing.
- ✓ Rebalancing keeps the system resilient.

ANALOGY

Think of Kafka Topic as a Book.
Partitions are Chapters (P0, P1, P2).
Consumers are Readers.

A reader can read one chapter at a time, and multiple readers can read different chapters together.

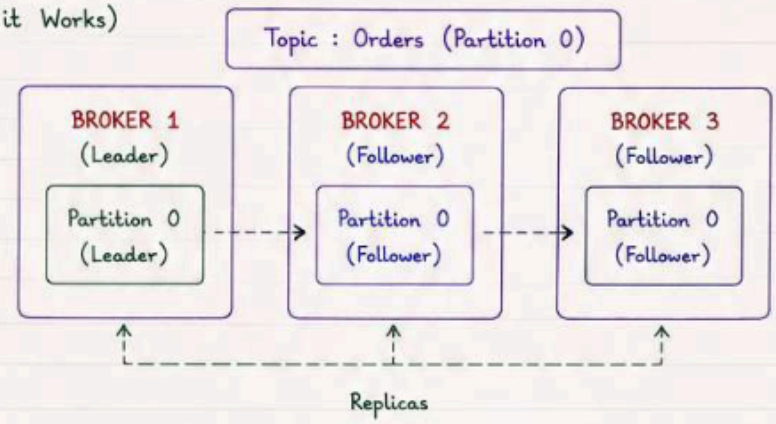


5. REPLICATION, FAULT TOLERANCE & DELIVERY GUARANTEES

Kafka is designed to be highly available and fault tolerant using Replication.

5.1 REPLICATION (How it Works)

- Each partition is replicated across multiple brokers.
- One broker acts as Leader.
- Others are Followers.
- If Leader fails, one Follower becomes new Leader.



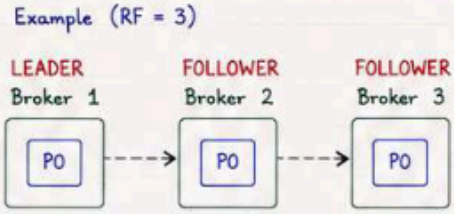
REPLICA FACTOR = 3

It means each partition has 3 copies.

Higher RF → More Availability
More Storage

5.2 ISR (IN-SYNC REPLICAS)

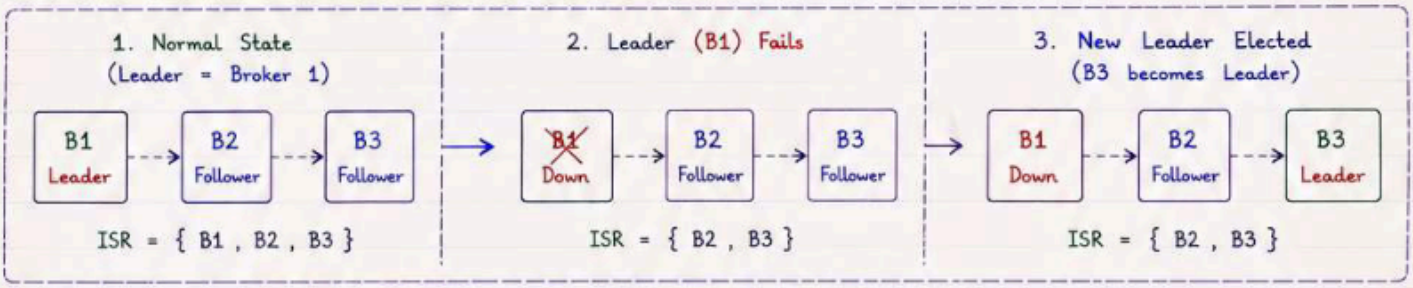
- ISR are replicas that are fully caught up with the Leader.
- Only ISR replicas can be new Leader.



ISR = { Broker 1 , Broker 2 , Broker 3 }

- If Broker 2 goes down → ISR = { Broker 1 , Broker 3 }
- If Leader goes down → Broker 3 (from ISR) becomes new Leader.

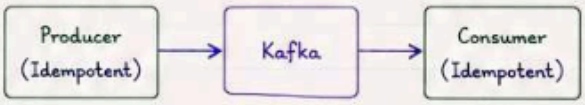
5.3 WHAT HAPPENS WHEN LEADER FAILS ?



5.4 DELIVERY GUARANTEES

GUARANTEE	DESCRIPTION	USE CASE
AT MOST ONCE	• Message may be lost but never processed more than once.	Logging, Metrics
AT LEAST ONCE	• Message will never be lost but may be processed more than once.	Emails, Notifications, Order Processing
EXACTLY ONCE	• Message will never be lost and never be processed more than once.	Payments, Financial Transactions

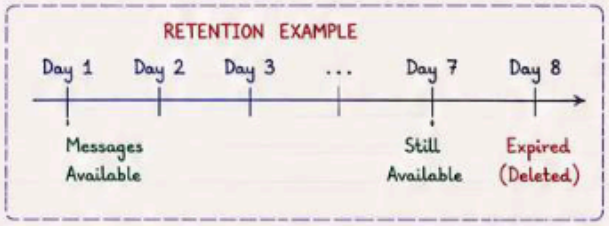
HOW EXACTLY ONCE WORKS (EOS)



- Idempotent Producer → Avoids duplicate messages.
- Transactions → Ensures atomicity (All or Nothing).
- Consumer reads only committed messages.

5.5 RETENTION & MESSAGE LIFETIME

- Kafka stores messages for a configurable amount of time.
- Defined by:
  - retention.ms (time based)
  - retention.bytes (size based)
- Default: 7 days (for most setups).



WHY RETENTION ?

- ✓ Reprocess data
- ✓ Replay events
- ✓ Audit & Compliance
- ✓ Consumer recovery

NOTE : Replication + ISR + Consumer Groups + Offsets together make Kafka highly available and fault tolerant.

→ More Replicas = More Safety = More Storage

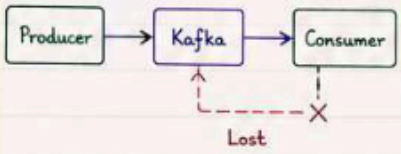
6. DELIVERY GUARANTEES, RETRY & DEAD LETTER QUEUE

Kafka provides different delivery guarantees depending on your use case.

6.1 DELIVERY GUARANTEES

① AT MOST ONCE

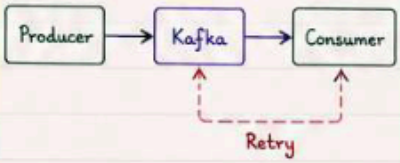
- Message may be lost
- But never delivered twice.



Use Case :
Logging, Metrics

② AT LEAST ONCE

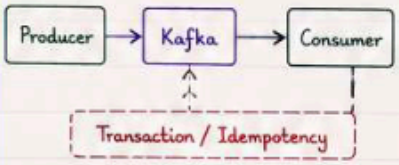
- Message will never be lost
- But may be delivered multiple times



Use Case :
Emails, Notification, Order Processing

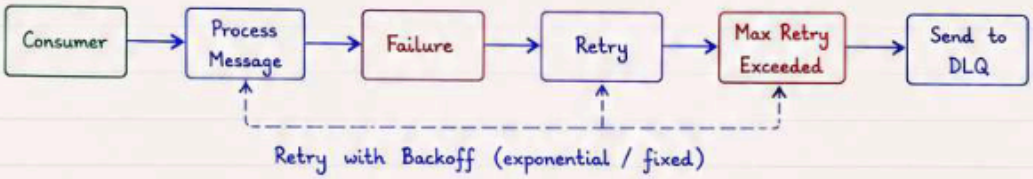
③ EXACTLY ONCE

- Message will never be lost
- And never be delivered twice.



Use Case :
Payments, Financial Transactions

6.2 RETRY MECHANISM (CONSUMER SIDE)



RETRY EXAMPLE FLOW

① Message consumed
② Processing fails
③ Retry 1 → fail
④ Retry 2 → fail
⑤ Retry N → fail
⑥ Send to DLQ

6.3 DEAD LETTER QUEUE (DLQ)

- Messages that fail after multiple retries are sent to a separate Topic (DLQ).
- Helps in manual analysis & reprocessing.



SPRING BOOT CONSUMER RETRY + DLQ (application.yml)

```yaml
spring:
  kafka:
    consumer:
      bootstrap-servers: localhost:9092
      group-id: order-group
      auto-offset-reset: earliest
    listener:
      ack-mode: manual_immediate
      type: batch
    properties:
      spring.json.trusted.packages: "*"
      spring.deserialization.value.delegate.class:
        org.springframework.kafka.support.serializer.JsonDeserializer
```

RETRY & DLQ CONFIG (application.yml)

```yaml
spring:
  kafka:
    listener:
      retry:
        enabled: true
        max-attempts: 3
        backoff:
          initial-interval: 2000
          multiplier: 2.0
          max-interval: 10000
      dead-letter-topic: orders.dlq
      dead-letter-republish: true
```

6.4 PRODUCER RELIABILITY SETTINGS

PROPERTY	VALUE	DESCRIPTION
acks	all	Leader + all ISR replicas must acknowledge.
retries	Integer.MAX_VALUE	Retry until successful.
enable.idempotence	true	Ensures exactly-once for producer.
max.in.flight.requests.per.connection	5 (default)	Use 1 for strict ordering with idempotence.

BEST PRACTICES

- ✓ Use meaningful keys.
- ✓ Choose right number of partitions.
- ✓ Enable retries with backoff.
- ✓ Use DLQ for better error handling.
- ✓ Monitor lag, consumer health.
- ✓ Make consumers idempotent for exactly-once.

QUICK SUMMARY

At Most Once	→ Fast, may lose data	Retry + Backoff	→ Handle temporary failures
At Least Once	→ Safe, may duplicate	DLQ	→ Handle permanent failures
Exactly Once	→ Safe + No duplicates	Replication	→ High availability & durability

Choose the right guarantee based on your business requirement!

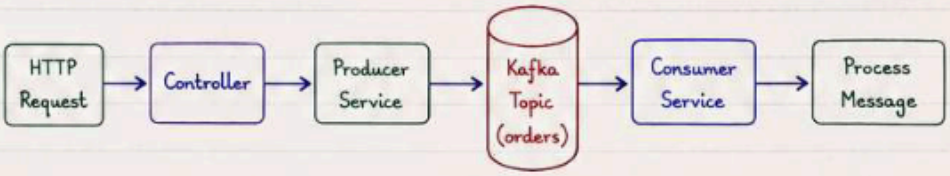
7. KAFKA IN SPRING BOOT

Let's build a simple Spring Boot application with Kafka.

7.1 PROJECT STRUCTURE

```
src
└ main
   └ java
      └ com.example.kafka
         ├ controller
         │  └ MessageController.java
         ├ producer
         │  └ KafkaProducerService.java
         ├ consumer
         │  └ KafkaConsumerService.java
         └ config
            └ KafkaConfig.java
└ resources
   └ application.yml
```

7.2 FLOW DIAGRAM



Technology Used

- Spring Boot
- Spring for Apache Kafka
- Apache Kafka

7.3 PRODUCER EXAMPLE

① application.yml (Producer Config)

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        spring.json.add.type.headers: false
```

② Producer Service

```java
@Service
public class KafkaProducerService {
    @Autowired
    private KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void sendOrder(OrderEvent order) {
        kafkaTemplate.send("orders", order.getOrderId(), order);
    }
}
```

③ Controller

```java
@RestController
@RequestMapping("/api/orders")
public class MessageController {
    @Autowired
    private KafkaProducerService producerService;
    @PostMapping
    public String createOrder(@RequestBody OrderEvent order) {
        producerService.sendOrder(order);
        return "Order sent to Kafka !";
    }
}
```

NOTE :

→ We are sending OrderEvent object to Kafka Topic 'orders'.
→ Key = orderId (helps in ordering in partition).
→ Value = OrderEvent (our payload).

7.4 CONSUMER EXAMPLE

① application.yml (Consumer Config)

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: order-group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframmert.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "*"
        auto-offset-reset: earliest
        enable-auto-commit: false
```

② Consumer Service

```java
@Service
public class KafkaConsumerService {
    @KafkaListener(topics = "orders", groupId = "order-group")
    public void listen(OrderEvent order) {
        System.out.println("Received Order : " + order.getOrderId());
        // Process the order
    }
}
```

@KafkaListener PARAMETERS

- topics : Kafka topic name
- groupId : Consumer group
- containerFactory : (Optional) custom factory
- autoStartup : (Optional) start listener automatically or not

7.5 ORDER EVENT (SAMPLE)

```java
public class OrderEvent {
    private String orderId;
    private String userId;
    private double amount;
    // getters and setters
}
```

Flow Summary

1. User calls REST API
2. Controller → Producer
3. Producer sends to Kafka
4. Consumer receives message
5. Business Logic is executed

KEY TAKEAWAYS

- ✓ Easy integration with Spring Boot
- ✓ Asynchronous communication
- ✓ Loose coupling between services
- ✓ Highly scalable
- ✓ Reliable message delivery
- ✓ Supports real-time event streaming

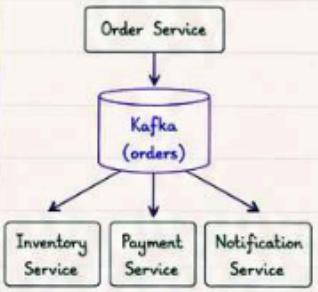
Spring Boot + Kafka = Perfect combination for modern microservices !

8. REAL WORLD USE CASES & BEST PRACTICES

8.1 REAL WORLD USE CASES

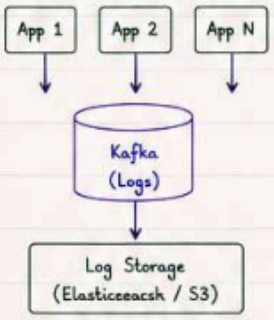
Kafka is used where we need high throughput, scalability and real-time data streaming.

① E-COMMERCE ORDER PROCESSING



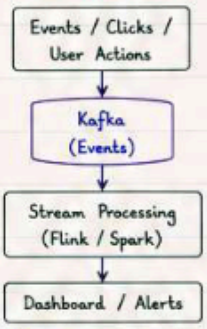
Order placed → Event → All services consume and process.

② LOG AGGREGATION



Centralized log collection for monitoring & analysis.

③ REAL-TIME ANALYTICS



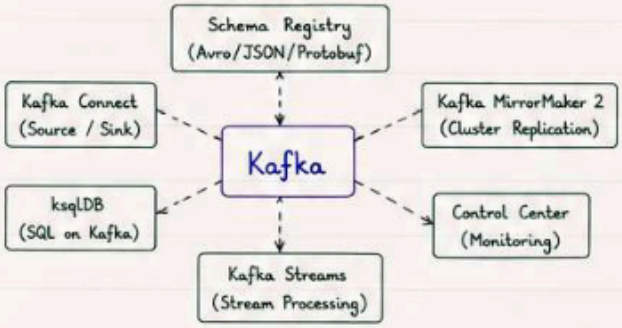
Real-time insights and alerts from streaming data.

④ EVENT SOURCING



All changes are stored as events for replay.

8.2 KAFKA ECOSYSTEM



8.3 MESSAGE FLOW SUMMARY

Producer Creates Message → Kafka Stores in Partitions → Replicated Across Brokers → Consumer Reads from Partitions → Offset Committed to Group

- Messages are immutable and appended to log.
- Consumers read at their own pace.
- Offsets track what has been consumed.
- Retention policy decides how long data is kept.

8.4 BEST PRACTICES

1. PRODUCER BEST PRACTICES

- ★ Use meaningful keys for better partitioning.
- ★ Enable acks=all for durability.
- ★ Enable retries with appropriate backoff.
- ★ Enable idempotence=true to avoid duplicates.
- ★ Use compression (snappy/lz4) for better throughput.

2. CONSUMER BEST PRACTICES

- ★ Use Consumer Groups for parallelism.
- ★ Commit offsets properly.
- ★ Handle failures with retry and DLQ.
- ★ Set max.poll.records and max.poll.interval.ms wisely.
- ★ Make consumers idempotent.

3. TOPIC & PARTITION BEST PRACTICES

- ★ Choose right number of partitions (not too less / not too many).
- ★ Use keys to keep related data in same partition (ordering).
- ★ Monitor partition skew.
- ★ Replication factor >= 3 for production.

4. OPERATIONAL BEST PRACTICES

- ★ Monitor broker, lag, and throughput.
- ★ Set proper retention policy.
- ★ Use separate topics for different use cases.
- ★ Backup critical data.
- ★ Regularly clean up old topics.

8.5 KAFKA vs RABBITMQ (QUICK COMPARISON)

FEATURE	KAFKA	RABBITMQ
Model	Event Streaming Platform	Traditional Message Queue
Data	Retained (Log)	Deleted after consumption.
Throughput	Very High (Millions msg/sec)	Lower compared to Kafka
Scalability	Horizontal (Partitions)	Limited (Queue based)
Use Case	Real-time streaming, analytics, logs, event sourcing	Task queues, RPC, simple messaging
Storage	Disk based (high durability)	Memory + Disk
Ordering	Per partition	Per queue

8.6 KAFKA CHEAT SHEET

- ▸ Broker → Kafka server.
- ▸ Topic → Logical channel.
- ▸ Partition → Split of topic for parallelism.
- ▸ Offset → Position of message in partition.
- ▸ Leader → Handles reads and writes.
- ▸ Follower → Replicate data from leader.
- ▸ ISR → In-Sync Replicas.
- ▸ Consumer Group → Group of consumers.
- ▸ DLQ → Dead Letter Queue.
- ▸ Retention → How long data is kept.
- ▸ Replication → Data copied across brokers.

☆ FINAL NOTES

- ✓ Kafka is a Distributed, Fault Tolerant, Scalable Event Streaming Platform.
- ✓ It is not just a message queue, it is a log system.
- ✓ Design topics, partitions and consumer groups carefully.
- ✓ Monitor, test failure scenarios and handle retries.
- ✓ Keep it simple, reliable and observable.

Understand the flow, Handle failures gracefully And build systems that scale with confidence !

REMEMBER

- ✓ Data never disappears in Kafka (until retention).
- ✓ Consumers read at their own speed.
- ✓ High throughput + High reliability.
- ✓ The heart of Modern Data Architecture !

★ Think in Events → Stream in Kafka → Act in Real-time ☆